Università
di Genova

High Performance Computing

Report Mandelbrot Algorithm

Andrea Cattarinich 5137057

November 27, 2025

Contents

1 Architectures

For this project I have used three workstation, my personal one, the workstation provided by the University and Google Colab.

1.1 Personal Workstation

HP Envy 17-ae103nl

CPU	
Model	Intel Core i7-6500U @ 2.50 GHz
Architecture	x86_64
Physical cores	2
Logical threads	4 (Hyper-Threading)
GPU	
Model	NVIDIA GeForce GTX 950M
CUDA Compute Capability	5.0
Driver version	572.16
CUDA Version	12.8
Software and Setup	
Visual Studio	Visual Studio Community 2022 (C++ Desktop workload)
CUDA Toolkit	Installed manually
WSL2	Enabled
NVIDIA Drivers	Version 572.16 (for WSL2)

1.2 University Workstation

Room 210

CPU	
Model	Intel Core i9-12900K (12th Gen)
Architecture	x86_64
CPU(s)	24
Threads per core	2
Cores per socket	16
Socket(s)	1
Max frequency	4.02 GHz
Profiling tools	Intel VTune Profiler, Intel Advisor

To use the Intel oneAPI tools, the environment was loaded with:

```
source /opt/intel/oneapi/setvars.sh
```

1.3 Google Colab

CPU	
Model	Intel(R) Xeon(R) CPU @ 2.20GHz
Architecture	x86_64
Physical cores	1
Logical threads	2 (Hyper-Threading)
Socket(s)	1
GPU	
Model	NVIDIA Tesla T4
CUDA Compute Capability	7.5
Driver version	550.54.15
CUDA Version	12.8

2 Hotspot Analysis

The Fourier Transform is a fundamental mathematical tool in digital signal processing. It converts a sequence of values from the **time domain** into a sequence of values in the **frequency domain**. This transformation makes it possible to identify which frequencies are present in a signal and with what intensity.

The continuous-time Fourier Transform is defined as:

$$X(\omega) = \int_{-\infty}^{+\infty} x(t)\, e^{-j\omega t}\, dt$$

The example shown in Figure 1 illustrates the practical and optical interpretation of the Fourier Transform of a rectangular signal, showing how a simple time–domain shape corresponds to a characteristic frequency–domain spectrum.

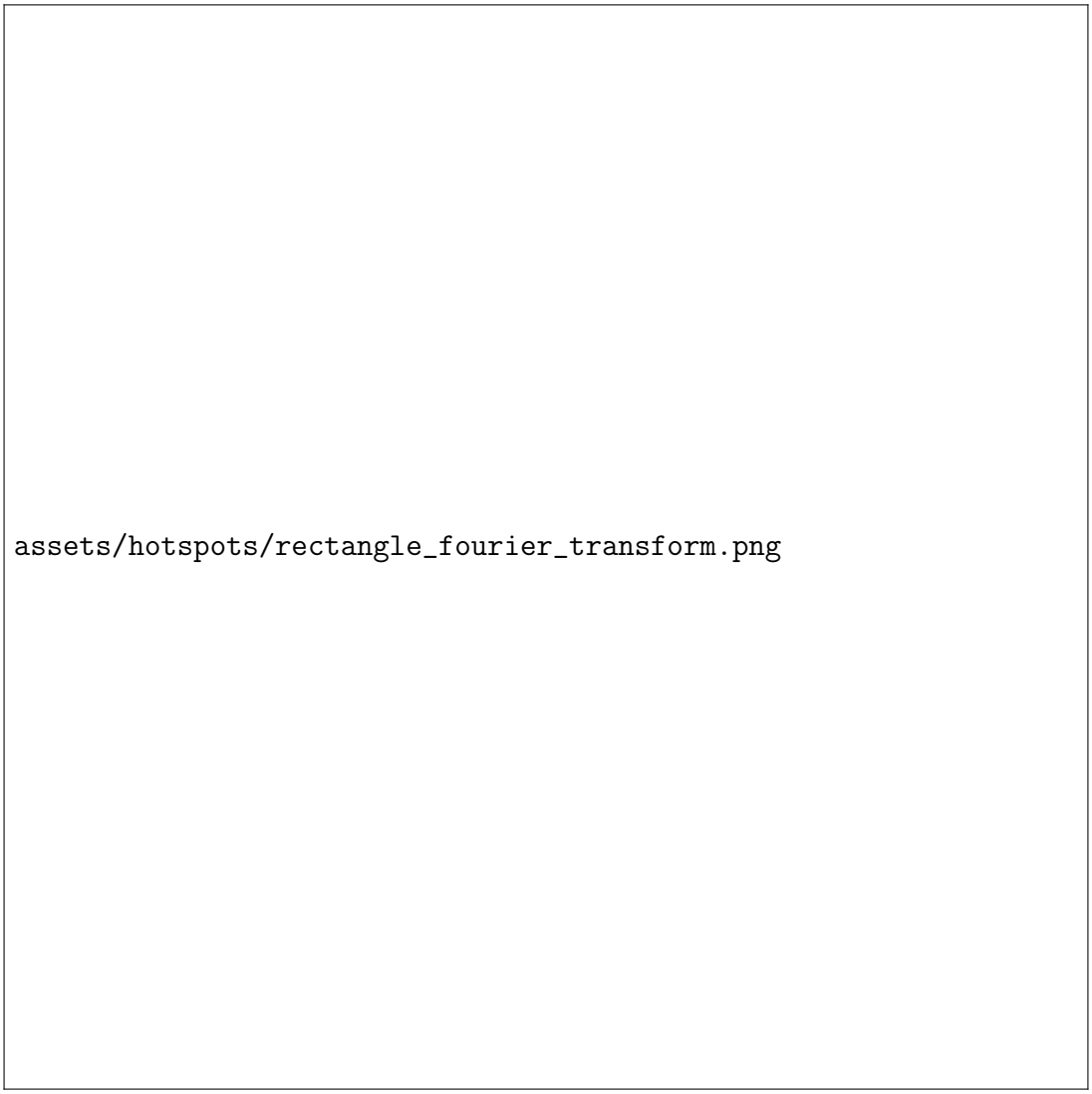


Figure 1: Fourier Transform of a rectangular signal.

2.1 Code Description

The algorithm provided by the course implements both the Discrete Fourier Transform (DFT) and its inverse (IDFT) in C. It computes the transform of a signal and verifies the correctness by applying the IDFT, following the discrete-time Fourier Transform definition:

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j2\pi kn/N}, \quad k = 0, 1, \ldots, N-1$$

The DFT is implemented following the standard quadratic-complexity $O(N^2)$ formulation, with two nested loops over the frequency index k and the time index n. Each output element can be computed independently, which makes the algorithm naturally parallelisable.

```c
int main() {
    int N = 100000;
    double *xr, *xi, *Xr, *Xi;

    fillInput(xr, xi, N);              // initialize input signal
    zero(Xr, Xi, N);                   // clear output buffers

    DFT(+1, xr, xi, Xr, Xi, N);        // compute DFT
    DFT(-1, Xr, Xi, xr, xi, N);        // compute IDFT

    checkResults(xr, xi, Xr, Xi, N); // verify correctness

    return 0;
}

void DFT(int idft, double* xr, ..., double* Xi, int N) {
    for (int k = 0; k < N; k++)
        for (int n = 0; n < N; n++) {
            double a = 2*M_PI*n*k/N;
            Xr[k] += xr[n]*cos(a) + idft*xi[n]*sin(a);
            Xi[k] += -idft*xr[n]*sin(a) + xi[n]*cos(a);
        }

    if (idft == -1)
        for (int n = 0; n < N; n++) {
            Xr[n] /= N;
            Xi[n] /= N;
        }
}
```

Listing 1: Pseudo code in C

2.2 Project Structure

The `homework1` project is organized into four main directories:

- **build**: contains the compiled executables.

- **reports**: stores the reports generated with the `-qopt-report` flag.
- **src**: includes the source code files.
- **metrics**: scripts that run the executables in `build` and save performance metrics.

The first step to improving the performance of a program is profiling. The goal of this analysis is to identify the parts of the program that consume the most execution time. To achieve this, I considered a data size resulting in a sequential execution time of ~ 30 seconds, by setting $N = 60'000$.

From this point onward, each executable is empirically measured over 10 runs to obtain reliable results.

2.3 Sequential and Parallel Sections

In order to apply Amdahl's Law (discussed in more detail in the Chapter 3), we must identify the sequential and parallel parts of the code. I have empirically measured these sections by separating the serial and parallel portions. More details of how I obtained these results can be found in the code `sequential_parallel_code_identification.c`.

The results are the following:

Quantity	**Value**
Total execution time T_{tot}	32.48231 s
Time of the parallelizable section T_{par}	32.48021 s
Time of the serial section $T_{\text{ser}} = T_{\text{tot}} - T_{\text{par}}$	0.00209 s
Fraction of serial code $f_s = \frac{T_{\text{ser}}}{T_{\text{tot}}}$	0.006 %

These values will be used to estimate the theoretical speedup according to Amdahl's Law:

$$S(p) = \frac{1}{f_s + \frac{1-f_s}{p}}\Big|_{f_s=0.00006} = \frac{1}{0.00006 + \frac{1-0.00006}{p}} \approx p$$

Note: the smaller the serial fraction f_s, the higher the potential speedup achievable by parallelizing the main loop of the DFT.

Note: in the real case, the speedup is not linear. In particular, from the parallelization examples presented in Chapter 5, the speedup is approximately linear up to 8 threads; beyond this point, it deviates from linear behavior.

2.4 Compilation

Since the Intel(R) C++ Compiler Classic (ICC) is deprecated and was removed from product release in the second half of 2023. The Intel(R) oneAPI DPC++/C++ Compiler (ICX) was used to performe the analysis.

To identify the optimal sequential compilation strategy, I conduced testing across multiple Intel compiler variants (`ICC` and `ICX`), focusing on the vectorization report provided by

the flag `-qopt-report`.
The program was compiled using the Intel compiler `ICX` with the following command (more details in Chapter 4):

```
icx -O2 -vec -g \
    -fopenmp \
    -qopt-report=max \
    -qopt-report-file=reports/omp_homework.optrpt \
    -o build/omp_homework \
    src/omp_homework.c
```

The compilation flags have the following meaning:

- `-O2`: optimizations for better performance;
- `-fopenmp`: activates compilation for OpenMP directives;
- `-vec`: activates automatic vectorization of loops using SIMD instructions;
- `-g`: includes debugging information in the executable;
- `-qopt-report=max`: generates a detailed optimization report;
- `-qopt-report-file=...`: specifies the file where the optimization report is saved;
- `-o build/omp_homework`: sets the name and location of the compiled executable;
- `src/omp_homework.c`: specifies the source file to compile.

2.5 Hotspot Identification

The measurements were performed using two methods: **Manually** (using `omp_get_wtime()`) and **Profiler** (using `Intel VTune`)

Table 1: Manually

Function	Time (s)	Percentage (%)

Table 2: Intel VTune Profiler

Function Stack	CPU Time: Total	Percentage (%)
"Total"	"69.97"	100.00
" $_{s}tart$"	"69.97"	100.00
" $_{i}ibc_{s}tart_{m}ain_{i}mpl$"	"69.97"	100.00
" main"	"69.97"	100.00
" std::abs¡double¿"	"41.2121"	58.90
" std::operator+¡double¿"	"13.6361"	19.49
" std::pow¡double¿"	"10.712"	15.31
" std::complex¡double¿::complex"	"0.0199997"	0.03

2.6 Conclusions

The main hotspot is the DFT() function and its inverse IDFT(), which together account for over 99,05% of the total execution time. All other functions have negligible impact. This indicates that **parallelization and vectorization efforts should focus on the double loop inside DFT()**.

The main hotspot is the loop contained in lines 72-80 (source: `src/omp_homework.c`). The dominant operations contributing to the algorithm's computational intensity are the `sin()` and `cos()` trigonometric functions, which account for the majority of the execution time, as shown in the following code.

3 Amdahl's Law

This section discusses Amdahl's Law, a fundamental principle in parallel computing that helps predict the theoretical speedup in latency when using multiple processors.

3.1 Performance Metrics

Considering T_p the time to solve a problem using p processors, we define:

- **Speedup**: $S(p) = \frac{T_1}{T_p},\ 0 < S(p) \leq p$
- **Efficiency**: $E(p) = \frac{S(p)}{p},\ 0 < E(p) \leq 1$

3.2 The Law

Amdahl's Law can be expressed as a classical formula:

$$S(p) = \frac{1}{f_s + \frac{(1-f_s)}{p}}$$

where:

- $S(p)$: is the theoretical speedup
- f_s: is the proportion of sequential code (non-parallelizable)
- $1 - f_s$: is the proportion of parallelizable code
- p: is the number of processors

Ideal case $(f_s = 0)$

Implies that all code is parallelizable. The speedup becomes:

$$S(p) = \frac{1}{\frac{1}{p}} = p$$

Maximum speedup equals the number of processors. Each processor contributes fully to the speedup.

Theoretical limit case $(p \to \infty)$

By increasing the number of processors to infinity, the speedup reaches its theoretical limit:

$$\lim_{p \to \infty} S(p) = \frac{1}{f_s}$$

Even tough the number of processors increases indefinitely, the speedup is limited by the sequential portion of the code, encountering a bottleneck.

Real case $(f_s = \alpha p)$

Where α is a constant $\alpha <= 0.1$ representing the overhead per processor. The speedup becomes:

$$S(p) = \frac{1}{\alpha p + \frac{1-\alpha p}{p}} = \frac{p}{1 - \alpha p + \alpha p^2}$$

There is no free lunch: as we add more processors, the overhead increases linearly, impacting the overall speedup.

For α small: when α is small we do not have a significant overhead.

$$S(p) \approx p$$

For α large: when α is large, the overhead dominates. The more we add processors, the less benefit we get!

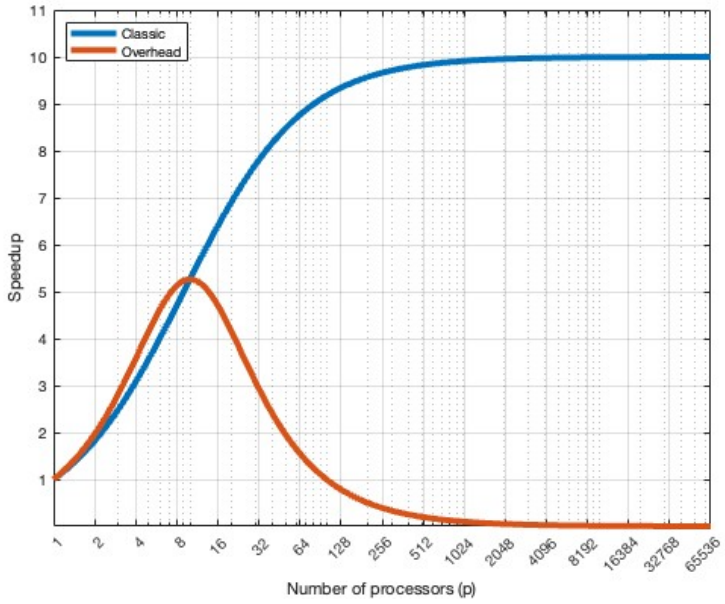
$$S(p) \approx \frac{p}{\alpha p^2} \approx \frac{1}{\alpha p}$$

3.3 Results

The following examples have been obtained using Matlab scripts are available in the repository (`classic_vs_overhead.m` and `parallel_comparison.m`).

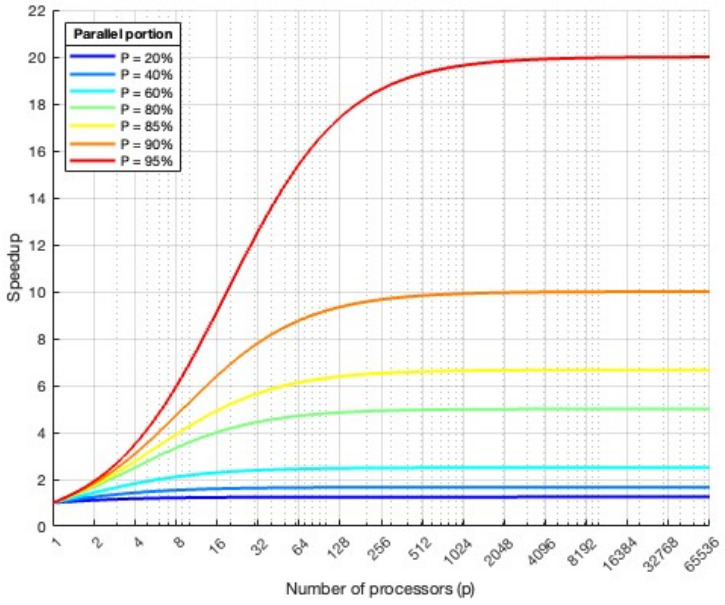
Classical vs Overhead

- The proportion of the parallelizable is 95% of code.
  ($f_s = 0.05$, $f_p = 1 - f_s = 0.95$)
- Overhead per processor
  $\alpha = 0.01$



Parallel speedup comparison

It is useful for understanding the limits of parallelization according to Amdahl's law: it highlights that simply increasing the number of processors is not enough if the sequential portion of the code is significant


Parallel portion
P = 20%
P = 40%
P = 60%
P = 80%
P = 85%
P = 90%
P = 95%
Speedup
Number of processors (p)
0
2
4
6
8
10
12
14
16
18
20
22
1
2
4
8
16
32
64
128
256
512
1024
2048
4096
8192
16384
32768
65536

4 Vectorization

The vectorization of the baseline code (`src/omp_homework.c`) was analyzed using Intel Advisor, focusing on performance hotspots and Roofline Analysis. The workflow included adding the executable and source folders, selecting **Roofline Analysis**, checking **Trip Counts**, and running the analysis.

4.1 Compilation Experiments through ICX

1. Baseline Compilation (no optimization)

Compilation flags: `-g -fopenmp`

Hotspots are memory-bound (DRAM), with no vectorization applied. Roofline shows performance limited by the Scalar Add Peak ($\sim$ 7.97 GFLOPS).

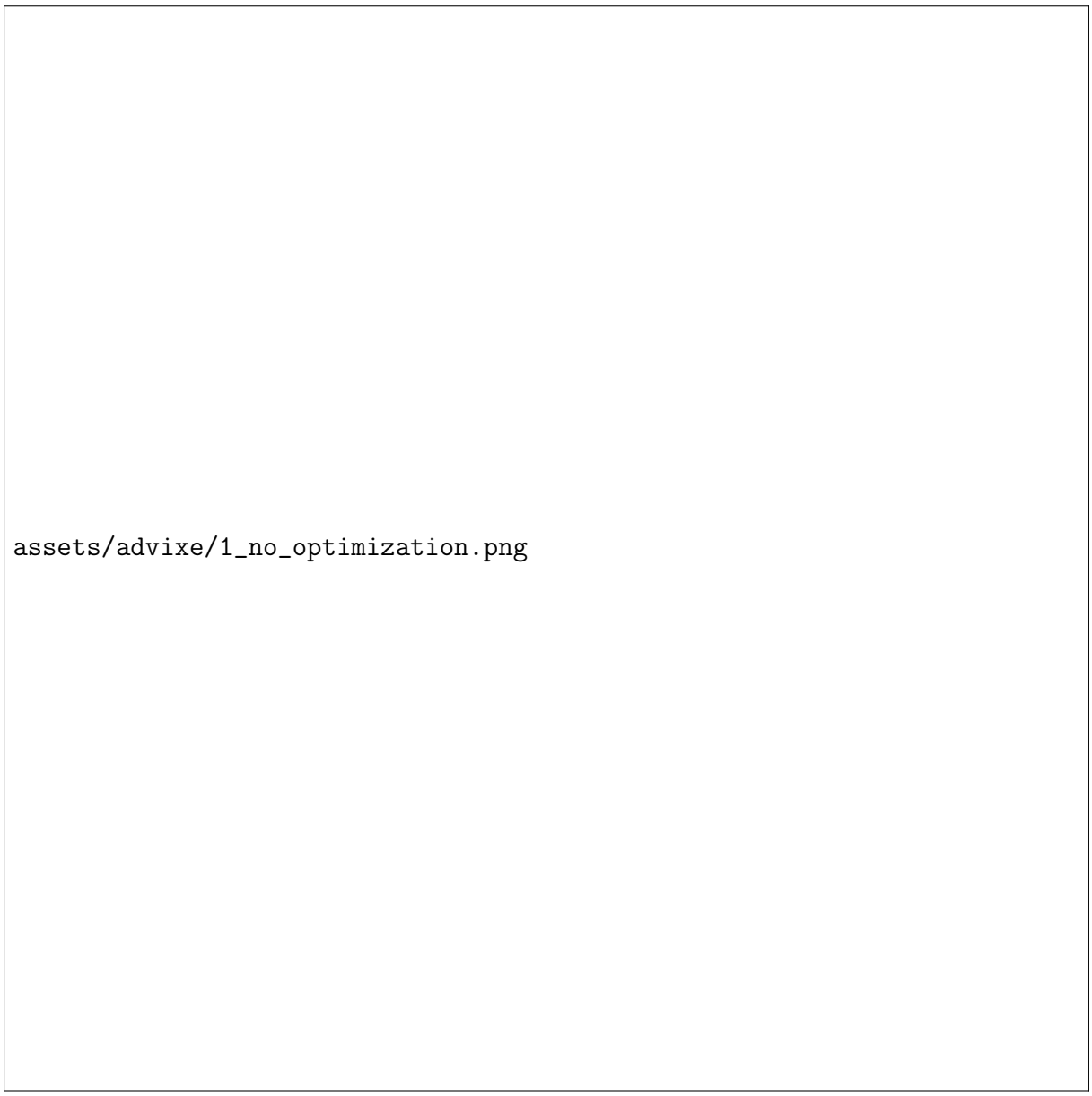


Figure 2: Roofline Analysis - baseline compilation.

2. Basic Optimization

Compilation flags: `-O0 -g -fopenmp`

Arithmetic intensity improves from 0.054 to 0.213 FLOP/byte. DRAM-bound, but performance increases relative to baseline. Hotspots correspond to trigonometric function calls.

Figure 3: Roofline Analysis - basic optimization.

3. Best-Optimized Compilation

Compilation flags: `-O2 -vec -g -fopenmp -xHost`

SIMD vectorization applied to critical loops. Performance rises from 10.1 GFLOPS to 34.5 GFLOPS while arithmetic intensity remains similar.

Figure 4: Roofline Analysis - best-optimized compilation.

4. Parallelization with Optimization

OpenMP parallelization over the outer loop (discussed in Chapter 5) further boosts performance. Roofline Analysis indicates:

Function	Arithmetic Intensity	Performance (GFLOPS)
cos	0.213	214,817
sin	0.232	418,012

Table 3: Performance of trigonometric functions with parallelization.

4.2 Conclusion

Roofline Analysis and Intel Advisor guided the optimization process:

Figure 5: Roofline Analysis - parallelized and optimized code.

- Baseline code is memory-bound with no vectorization.
- Compiler optimizations and architecture-specific flags increase performance and apply SIMD efficiently.
- Combining vectorization with OpenMP parallelization allows computationally intensive functions to approach hardware limits.

5 Parallelization

5.1 Parallelization over the outer loop

In the first approach (`src/omp1.c`), the outer loop over the frequency index k is parallelized.

```
#pragma omp parallel for private(n)
for (k = 0; k < N; k++) {
    double Xr_temp = 0.0;
    double Xi_temp = 0.0;
    for (n = 0; n < N; n++) {
        Xr_temp += xr[n] * cos(n * k * PI2 / N) + idft * xi[n] *
           sin(n * k * PI2 / N);
        Xi_temp += -idft * xr[n] * sin(n * k * PI2 / N) + xi[n] *
            cos(n * k * PI2 / N);
    }
    Xr_o[k] = Xr_temp;
    Xi_o[k] = Xi_temp;
}
```

Here, each thread computes a subset of the output frequencies independently. The temporary variables `Xr_temp` and `Xi_temp` are private to each thread, ensuring there is no race condition when updating the output arrays.

5.2 Parallelization over the inner loop with reduction

In the second approach, the inner loop over the time index n is parallelized using a reduction clause:

```
for (k = 0; k < N; k++) {
    double Xr_temp = 0.0;
    double Xi_temp = 0.0;
#pragma omp parallel for reduction(+:Xr_temp, Xi_temp)
    for (n = 0; n < N; n++) {
        Xr_temp += xr[n] * cos(n * k * PI2 / N) + idft * xi[n] *
           sin(n * k * PI2 / N);
        Xi_temp += -idft * xr[n] * sin(n * k * PI2 / N) + xi[n] *
            cos(n * k * PI2 / N);
    }
    Xr_o[k] = Xr_temp;
    Xi_o[k] = Xi_temp;
}
```

Here, the computation of each output frequency $X[k]$ is parallelized across the time samples n. The `reduction` clause ensures that each thread accumulates contributions to `Xr_temp` and `Xi_temp` safely, avoiding race conditions. This method can provide higher parallel efficiency for large N, but incurs some overhead from the reduction operation, as shown in the following figures.

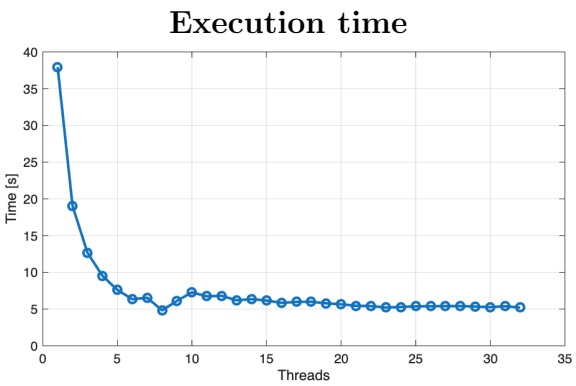
5.3 Results

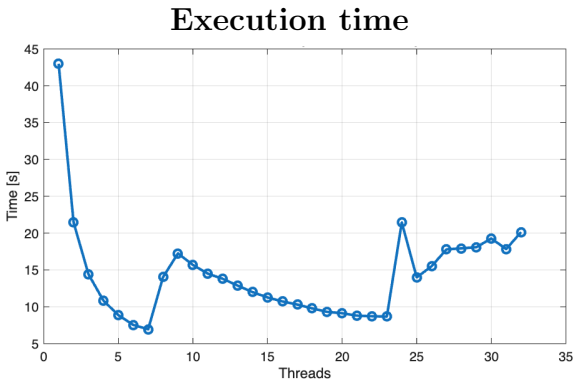
Parallelized `for` loop with OpenMP using `private(n)` to make each thread independent.

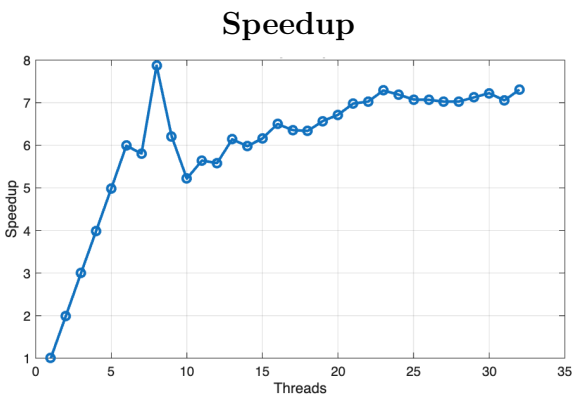
```
#pragma omp parallel for private(n)
for (k=0 ; k<N ; k++)
{
     double Xr_temp = 0.0;
     double Xi_temp = 0.0;

     for (n=0 ; n<N ; n++)  {
          Xr_temp += ...
          Xi_temp += ...
     }

     Xr_o[k] = Xr_temp;
     Xi_o[k] = Xi_temp;
}
```

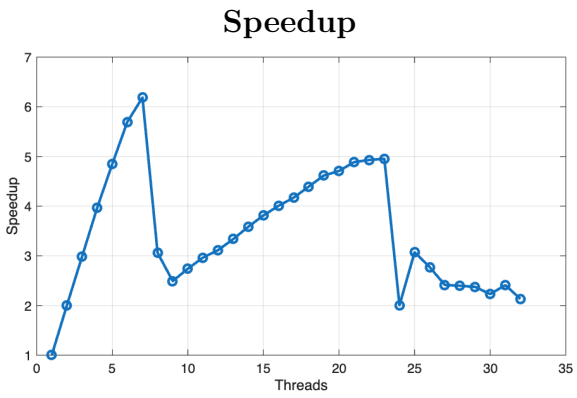
Parallelized `for` loop with OpenMP using `reduction` to combine the results from all threads.

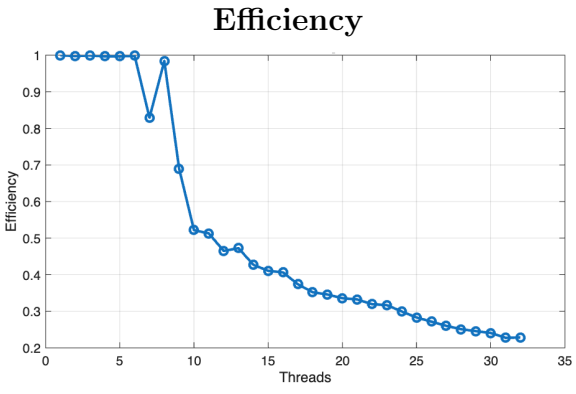
```
int k, n;
for (k=0 ; k<N ; k++)
{
     double Xr_temp = 0.0;
     double Xi_temp = 0.0;

#pragma omp parallel for reduction(+:
     Xr_temp, Xi_temp)
     for (n=0 ; n<N ; n++)  {
          Xr_temp += ...
          Xi_temp += ...
     }

     Xr_o[k] = Xr_temp;
     Xi_o[k] = Xi_temp;
}
```

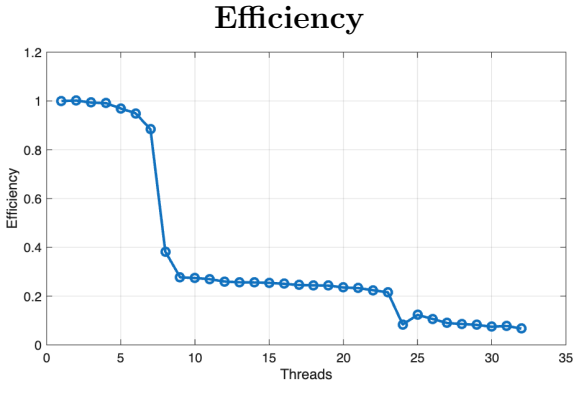












Attempted Optimization

Precomputation of Trigonometric Functions: in an additional attempt to optimize the DFT computation, I modified the code as in the source `src/omp3.c`. The idea was to reduce the repeated evaluation of trigonometric functions by computing the sine and cosine values once per iteration:

```
#pragma omp parallel for private(n)
for (k=0; k<N; k++) {
    double Xr_temp = 0.0;
    double Xi_temp = 0.0;

#pragma omp simd reduction(+:Xr_temp, Xi_temp)
    for (n=0; n<N; n++) {
        double angle = n * k * PI2 / N;
        double c = cos(angle);
        double s = sin(angle);

        Xr_temp += xr[n]*c + idft*xi[n]*s;
        Xi_temp += -idft*xr[n]*s + xi[n]*c;
    }

    Xr_o[k] = Xr_temp;
    Xi_o[k] = Xi_temp;
}
```

The motivation behind this modification was that, theoretically, computing the trigonometric functions only once per iteration should reduce the overall workload and improve performance.

However, the experiments did not yield significant improvements. Performance remained the same, or in some cases slightly worse. This is likely due to the compiler automatically applying this optimization by default.

6 Conclusions

Both parallelization approaches exhibit near-linear scaling with an efficiency of approximately 100% up to 8 cores. The first parallelization strategy demonstrated the best performance, achieving a maximum speedup of 7.87 on 8 cores. Beyond 24 cores, additional cores provided no significant improvement.